

CGV-Web: impacto das otimizações de renderização

Resultados do benchmark automatizado (RTX 4070 Ti, dados mesclados)

1 Metodologia

O desempenho foi medido por um *runner* externo ao aplicativo, que executa um laço `request-AnimationFrame` próprio e registra o timestamp bruto de cada quadro apresentado; o método é idêntico nas duas versões e não instrumenta o código medido. Ambas as versões rodaram na mesma máquina (NVIDIA RTX 4070 Ti, Chrome com limite de FPS destravado, *viewport* 1920×1080 , *devicePixelRatio* 1), sobre os mesmos sete eventos reais do ATLAS (23,3 mil a 57,4 mil células de calorímetro), em três cenários: *tour* (volta automática padronizada da câmera, 3×78 s por evento), *corte parado* e *arraste do corte* (8 s por eixo).

Mescla das execuções (agregação por réplica). Trata-se cada suíte completa como uma réplica: por cenário e versão calcula-se a estatística de cada execução (agrupando suas repetições) e agrega-se pela *média entre execuções*, com peso igual, o que evita favorecer execuções mais rápidas (que geram mais quadros) e expõe a variabilidade entre sessões. Nesta máquina entram no pool **três** execuções completas do *current* e **duas** do *baseline* (ciclo de 78 s); uma varredura rápida adicional do *baseline* (ciclo de 10 s) serve apenas de conferência de reprodutibilidade. As células efetivamente renderizadas conferem entre as versões (diferença de 128 células, até 0,65%, a favor da versão antiga) e os triângulos por quadro diferem cerca de 7%, também a favor da antiga — ambas conservadoras.

2 Resultados

Tabela 1: FPS mediano (média entre execuções), faixa do *current* entre execuções, *speedup* e *draw calls* por quadro. “b” = baseline (antiga), “c” = current (nova).

Cenário	Evento	Células	FPS b→c	Nova (faixa)	Speedup	Draws b→c
tour	517743.363	23 349	47,6 → 370	294–417	7,8×	13 835 → 46
tour	518084.443	23 612	47,6 → 355	294–400	7,5×	13 974 → 58
tour	518084.891	25 825	47,5 → 355	294–400	7,5×	14 064 → 66
tour	516761.342	39 043	46,3 → 318	263–357	6,9×	14 587 → 160
tour	520526.948	40 976	44,3 → 304	256–333	6,9×	15 243 → 195
tour	520249.978	43 367	44,7 → 316	256–357	7,1×	15 198 → 192
tour	520534.398	57 359	40,8 → 306	270–345	7,5×	16 860 → 185
corte parado	520534.398	57 359	41,5 → 289	250–313	7,0×	16 864 → 189
arraste θ	520534.398	57 359	8,3 → 242	213–263	29,0×	17 112 → 200
arraste z	520534.398	57 359	8,5 → 242	213–256	28,5×	17 370 → 192
arraste raio	520534.398	57 359	10,0 → 242	213–256	24,1×	17 339 → 199
Média geométrica dos <i>speedups</i> (11 cenários)					10,4×	

Visão geral. Agregando as cinco execuções completas, a versão nova é mais rápida em todos os 11 cenários comuns, com ganho típico (média geométrica) de **10,4×** no FPS mediano (Tabela 1, Figura 1). O ganho tem dois regimes. Na visualização passiva, o *tour* e o corte parado ficam entre 6,9× e 7,8× (média geométrica de 7,3× nos *tours* e 7,0× no corte parado): a versão antiga não atinge 60 FPS em nenhum evento (40,8 a 47,6 FPS), enquanto a nova opera entre 289 e 370 FPS de média, com 1% low de 236 a 291 FPS. No *arraste do corte*, o caso interativo crítico, o ganho vai de 24,1× a 29,0× (média geométrica de 27,1×): a antiga caía para 8 a 10 FPS medianos (1% low de 6 FPS; *frame-time* mediano de 98 a 120 ms), efetivamente inutilizável, enquanto a nova sustenta 242 FPS (Figura 4).

Mecanismo. A causa dominante é a eliminação do gargalo de CPU por *draw calls* (Figura 2): a antiga emite de 13,8 mil a 17,4 mil ordens de desenho por quadro, número que cresce com a quantidade de células do evento; a nova agrupa as células em lotes e emite de 46 a 200 (redução de duas ordens de grandeza), desenhando praticamente a mesma geometria. Com a CPU fora do caminho crítico, a vazão útil da GPU sobe de 20–108 para 565–881 Mtri/s, cerca de 10×. No arraste soma-se a mudança de algoritmo: o corte passou a ser aplicado na GPU, pela atualização de um parâmetro, em vez de recalculado na CPU a cada movimento do mouse.

Latência e regularidade. O ganho não é apenas de média; a cauda encolhe drasticamente. No *tour* do evento mais pesado, o p99 do *frame-time* cai de 29,5 ms para cerca de 4 ms. Nos arrastes, o p99 da antiga fica em 160–173 ms; o da nova, entre 18 e 19 ms de média entre execuções (algumas execuções mantêm a cauda em ~ 7 ms, outras a levam a ~ 40 ms), ainda muito abaixo da antiga. O eixo azimutal φ , inédito na versão nova e por isso sem baseline de comparação, mantém 232 FPS medianos com cauda maior (1% low de 67 FPS; p99 de 29 ms), associada a picos ao recruzar a emenda de φ em $\pm\pi$. Fora do laço de renderização, abrir um evento (análise do JiveXML) ficou tipicamente 5,4× mais rápido (o pior caso cai de 5,4 s para 0,6 s), ao custo da preparação da geometria ao iniciar o aplicativo, que subiu de $\sim 0,4$ s para $\sim 1,0$ s.

Reprodutibilidade entre execuções. As réplicas revelam que a versão antiga é muito estável entre execuções (as duas medições completas concordam dentro de $\sim 4\%$, e a varredura rápida de 10 s reproduz o mesmo patamar de ~ 45 FPS), ao passo que a versão nova, por rodar a centenas de FPS, é mais sensível a variações da máquina (carga de fundo, aquecimento): entre as três execuções completas o FPS mediano dos *tours* varia até 42% (256 a 417 FPS conforme o evento e a execução), como mostra a Figura 3. Por isso os valores deste relatório usam a média entre execuções, uma estimativa conservadora: mesmo a execução mais lenta da nova (256 FPS nos *tours*, 213 FPS no arraste) mantém o ganho em ~ 5 – $6\times$ na visualização e $\sim 25\times$ no arraste.

Ameaças à validade. (i) A carga não é bit-idêntica: a versão nova renderiza 128 células a mais (+0,24% a +0,65%) e cerca de 7% mais triângulos por quadro; ambas as diferenças pesam contra a versão nova. (ii) No arraste, o ângulo de abertura registrado do corte difere (180° na antiga, $110,2^\circ$ na nova); como células renderizadas e triângulos por quadro conferem, o efeito é de segunda ordem frente ao ganho de $24\times$ a $29\times$. (iii) O arraste tem uma repetição de 8 s por eixo, contra três repetições de 78 s nos *tours*, e sua cauda é a mais sensível entre execuções. (iv) GPU, navegador, resolução e trajetória de câmera idênticos, verificados nos metadados de todas as execuções.

3 Conclusão

Na mesma máquina e sob medição externa idêntica, mesclando cinco execuções completas, as otimizações (agrupamento de células em lotes e corte na GPU) elevam o CGV-Web de um regime

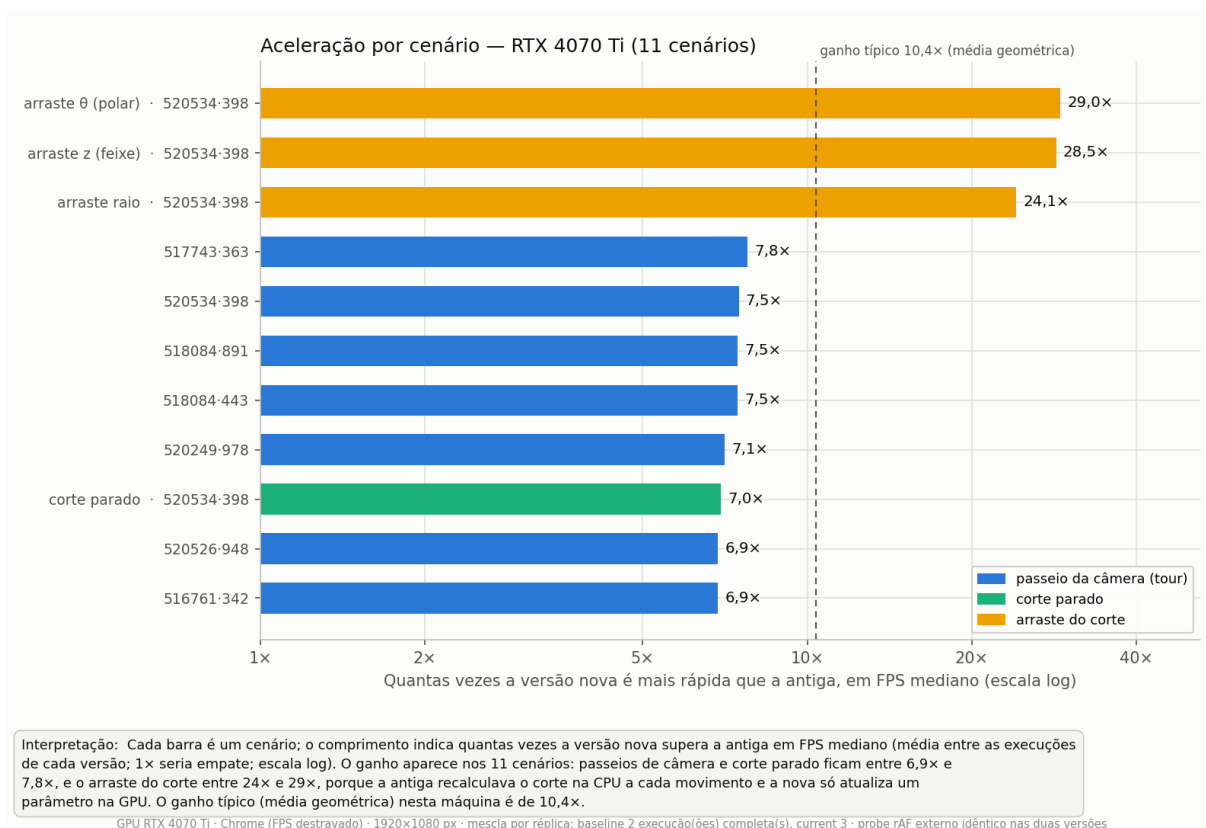


Figura 1: *Speedup* do FPS mediano por cenário (média entre execuções, escala log). Ganho típico de 10,4x; arrastes de corte entre 24x e 29x.

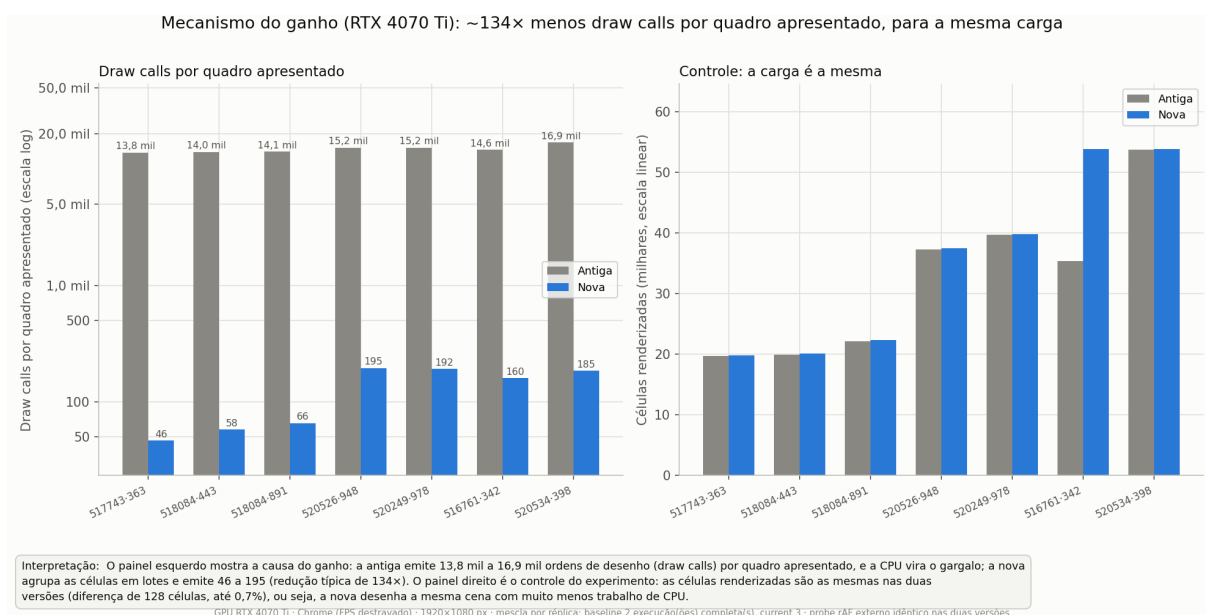


Figura 2: Mecanismo do ganho: duas ordens de grandeza menos *draw calls* por quadro nos *tours*, para a mesma carga de células.

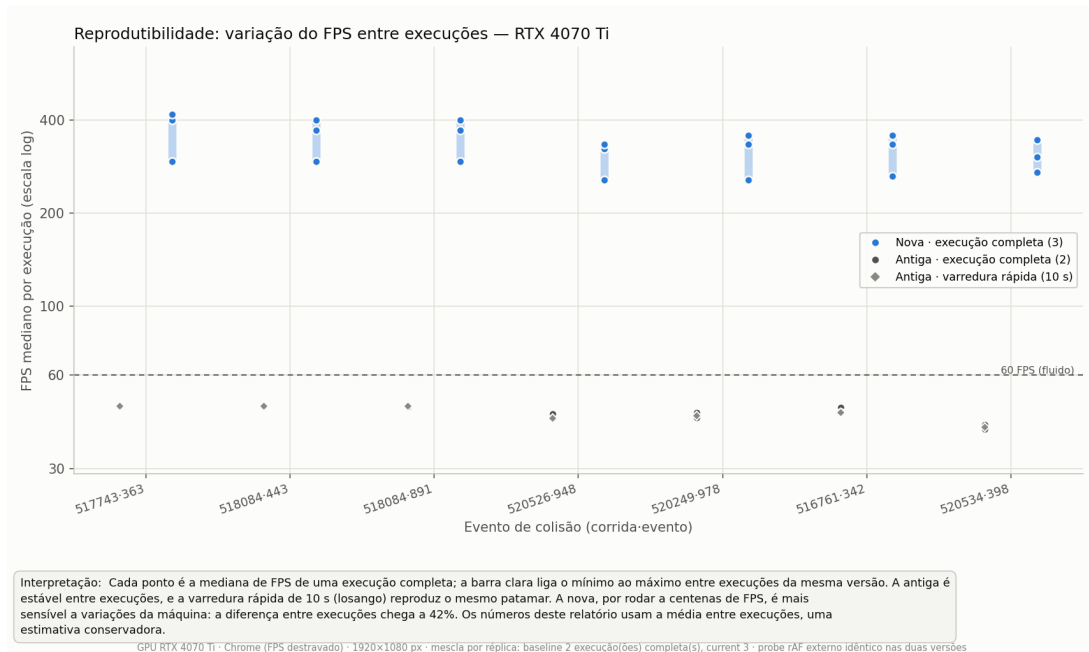


Figura 3: Variabilidade entre execuções: a antiga é estável (e a varredura rápida de 10 s a confirma), enquanto o FPS da nova varia até 42% entre execuções, permanecendo sempre muito acima de 60 FPS.

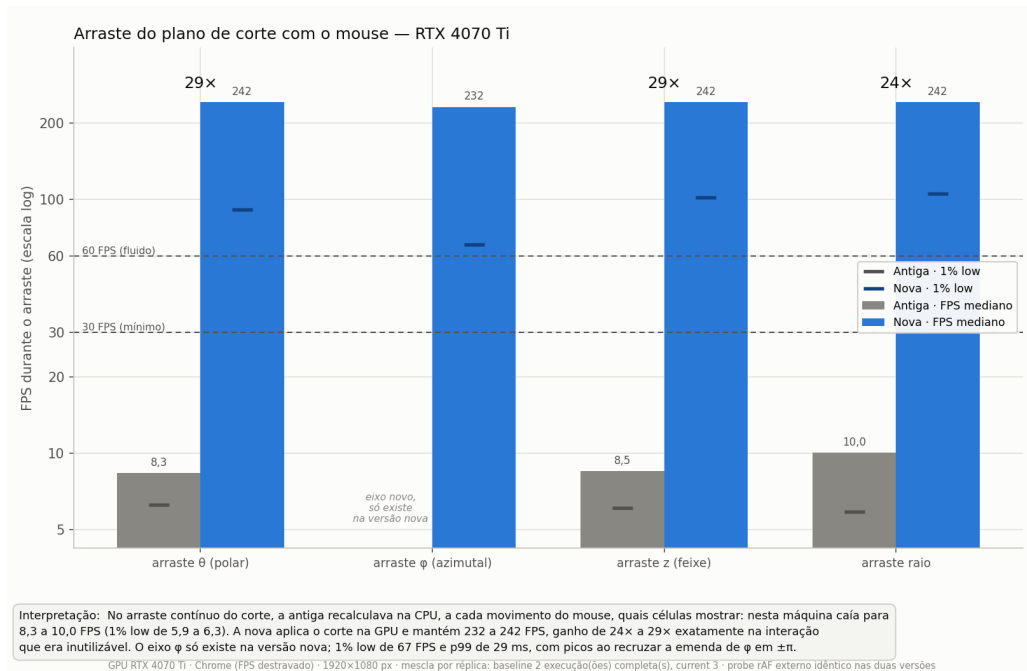


Figura 4: Arraste do plano de corte: de 8 a 10 FPS na versão antiga para 242 FPS na nova (média entre execuções). O eixo φ só existe na versão nova.

consistentemente abaixo de 60 FPS, com interação de corte a 8–10 FPS, para centenas de FPS em todos os cenários medidos: ganho típico de $10,4\times$ e de até $29\times$ na interação crítica. A versão nova apresenta maior variabilidade entre execuções (até 42%), esperada em um aplicativo que passou a rodar a centenas de FPS, mas até o seu pior caso preserva uma folga larga sobre a versão antiga e sobre o limiar de fluidez. Numa máquina modesta (GTX 1050 Ti) o mesmo experimento rende $37\times$ (relatório próprio), confirmando que o ganho cresce quanto mais a CPU domina o custo do quadro.